SUMMIT: A COLLABORATIVE MULTIPLAYER WEB IDE AND AI CODING AGENT WITH PERSISTENT PROJECT MEMORY

PHASE REPORT

Submitted by

MUHILAN S – 241701033

DHANESH SK – 241701013

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING IN

COMPUTER SCIENCE AND DESIGN





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this report titled "SUMMIT: A COLLABORATIVE MULTIPLAYER WEB IDE AND AI CODING AGENT WITH PERSISTENT PROJECT MEMORY" is the bonafide work of MUHILAN S and DHANESH S K, who carried out the work under my supervision. Certified further that, to the best of my knowledge, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Certified further that this project contributes to SDG 4 – Quality Education, SDG 8 – Decent Work and Economic Growth, SDG 9 – Industry, Innovation and Infrastructure, and SDG 10 – Reduced Inequalities, with a primary focus on SDG 9 – Industry, Innovation and Infrastructure, through the development of an innovative, collaborative AI-powered software development environment that promotes accessible, efficient, and inclusive software engineering.

Signature of the HOD

Dr. S. Vinodh Kumar
Professor,The Head of the Department
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai

Signature of the Supervisor

Dr. P. Revathy
Professor,
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai

Submitted for the Project Viva-Voce examination held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

Contemporary Large Language Model (LLM)-based coding assistants operate in **stateless, single-user environments,** often losing project context, architectural decisions, and team discussions between sessions. Existing development environments also lack **real-time collaboration with a shared autonomous AI coding agent.** This project presents **Summit**, a browser-native collaborative Web IDE and multiplayer AI coding agent platform designed to overcome these limitations through **persistent project memory and synchronized multi-user interaction.**

Summit incorporates a **dual-tier persistent memory system** using an ACID-compliant SQLite database to retain architectural decisions, coding conventions, task statuses, and conversation history across sessions. A **deterministic path-relevance context retriever** dynamically assembles relevant information into model prompts. The platform also integrates **Monaco Editor, Xterm.js, sandboxed live web previewing, and FastAPI-based WebSocket synchronization,** enabling multiple developers to collaborate with a unified AI agent. **Optimistic Concurrency Control (OCC)** detects conflicting file modifications and provides visual diff-based resolution to prevent destructive overwrites.

Built using **React 19, TypeScript, FastAPI, and LiteLLM**, Summit supports cloud-based models such as **Claude 3.5 Sonnet, GPT-4o, Gemini 2.5 Pro, and DeepSeek-V3**, alongside local open-weight models through Ollama. Phase I establishes the problem definition, comparative literature foundation, four-tier system architecture, relational schema, and complete module implementation. The system is validated through a **17-test functional verification suite** covering REST API contracts, 15 WebSocket event schemas, relational memory upserts, and workspace path isolation. Future enhancements include **keystroke-level CRDT co-editing, dense vector semantic retrieval, and containerized microVM sandboxing.**

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN**, **Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. S. VINODHKUMAR, Ph.D.**, Professor and Head of the Department of Computer Science and Design for his guidance and encouragement throughout the project work .We convey our sincere and deepest gratitude to our internal guide ,**DR.P.REVATHY.,** Department of Computer Science and Design , for her valuable guidance throughout the course of the project.

MUHILAN S(241701033)

DHANESH SK (241701013)

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
	BONAFIDE CERTIFICATE	ii
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
	LIST OF TABLES	v
	LIST OF FIGURES	vi
	LIST OF ABBREVIATIONS	vii
1	**INTRODUCTION**	**1**
	1.1 General	1
	1.2 Background	1
	1.3 Problem Statement	2
	1.4 Motivation	2
	1.5 Objectives	2
	1.6 Operational Boundaries and System Coverage	3
	1.7 Existing System	3
	1.8 Limitations of Existing System	4
	1.9 Proposed System	4
	1.10 Advantages of Proposed System	5
2	**LITERATURE REVIEW**	**6**
	2.1 Introduction	6
	2.2 Review of Related Work	6
	2.3 Comparative Analysis	8
	2.4 Comparative Analysis of the Proposed System	9
	2.5 Research Gap	10
3	**THEORETICAL BACKGROUND AND SYSTEM DESIGN**	**11**

CHAPTER	TITLE	PAGE NO.
	3.1 General	11
	3.2 Browser-Native IDE Rendering and Monaco Editor Core	11
	3.3 Asynchronous Bidirectional Terminal Emulation	11
	3.4 Sandboxed Live Web Application Previewing	12
	3.5 Full-Duplex WebSocket Communication Protocol	12
	3.6 Multiplayer Room Synchronization and Presence State Management	12
	3.7 ReAct (Reason and Act) Autonomous Agent Execution Pattern	13
	3.8 Unified Multi-Provider LLM Routing via LiteLLM	13
	3.9 Agent Tool Invocation Interface and Workspace Sandboxing	13
	3.10 Dual-Tier Relational Memory Architecture	14
	3.11 Deterministic Path-Relevance Context Retrieval and Prompt Assembly	14
	3.12 Optimistic Concurrency Control (OCC) and Monotonic Versioning	15
	3.13 Visual Conflict Resolution and Side-by-Side Diff Inspection	15
	3.14 Automated Git Checkpointing and Rollback Mechanics	15
	3.15 Relational Schema Design and ACID Transaction Guarantees	16
	3.16 FastAPI Asynchronous Orchestration Layer	16
	3.17 System Architecture	16
	3.18 Data Flow Diagram	18
	3.19 Activity Diagram	21
	3.20 Overall System Diagram	22
	3.21 Core Conceptual Classes and Database Schema	22
4	**SYSTEM IMPLEMENTATION AND PROJECT DESCRIPTION**	**25**
	4.1 General	25
	4.2 Requirement Analysis	25
	4.3 Functional Requirements	25

CHAPTER	TITLE	PAGE NO.
	4.4 Non-Functional Requirements	27
	4.5 Technology Selection	28
	4.6 System Modules	30
	4.7 Event Telemetry and Tool Action Mapping	32
	4.8 Module Integration	33
	4.9 System Configuration	34
	4.10 Feasibility Study	35
	4.11 Verification and Testing Strategy	36
	4.12 Initial Results	38
	4.13 Challenges	38
5	**CONCLUSIONS AND FUTURE ENHANCEMENTS FOR PHASE II**	**39**
	5.1 Conclusion	39
	5.2 Current Phase-I Status	39
	5.3 Limitations	39
	5.4 Phase-II Objectives	40
	5.5 Future Enhancements	40
	5.6 Phase-II Work Schedule	41
	CO–SDG RELEVANCE RECORD	42
	COURSE OUTCOMES	43
	PROGRAM OUTCOMES	44
	PROGRAM SPECIFIC OUTCOMES	45
	CO–PO–PSO MAPPING	46

LIST OF TABLES

Table No.	Title	Section Reference
Table 1.1	Limitations of existing developer environments and AI coding systems	Section 1.7
Table 2.1	Literature survey comparison across collaborative editing, code retrieval, and agentic systems	Section 2.3
Table 2.2	Comparative analysis of state-of-the-art developer systems versus Summit	Section 2.4
Table 3.1	Four-tier architectural specification and inter-tier communication protocols	Section 3.17
Table 3.2	Hierarchical Data Flow Diagram (Level 0, Level 1, and Level 2) process and store mapping	Section 3.18
Table 3.3	Core conceptual classes and service abstractions	Section 3.22
Table 3.4	Relational database schema definitions in summit.db	Section 3.22
Table 4.1	Functional requirements specification	Section 4.3
Table 4.2	Non-functional requirements specification	Section 4.4
Table 4.3	Full-stack technology stack	Section 4.5

Table No.	Title	Section Reference
Table 4.4	Technology selection rationale	Section 4.5
Table 4.5	System module descriptions and architectural responsibilities	Section 4.6
Table 4.6	Mapping of WebSocket events and autonomous agent tools to system actions	Section 4.7
Table 4.7	Hardware configuration for development and verification	Section 4.9
Table 4.8	Software and runtime environment configuration	Section 4.9
Table 4.9	Functional verification test suite (17 automated test cases)	Section 4.11.5
Table 5.1	Future enhancements and Phase-II engineering schedule	Section 5.6

LIST OF FIGURES

Visual Reference	Title	Section Reference
Visual 1	AI Pair Programming Halves Task Time While Repository Context Boosts Code Accuracy by Over 15 Percentage Points	Section 2.3
Visual 2	Phase-I Verification Baseline Achieved With Five Core Milestones Scheduled Across Phase-II	Section 5.6

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ACI	Agent-Computer Interface
ACID	Atomicity, Consistency, Isolation, Durability
AI	Artificial Intelligence
ANSI	American National Standards Institute
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
AST	Abstract Syntax Tree
CLI	Command Line Interface
CRDT	Conflict-Free Replicated Data Type
CSCW	Computer-Supported Cooperative Work
DFD	Data Flow Diagram
EM	Exact Match
ES	Edit Similarity
FR	Functional Requirement
HCI	Human-Computer Interaction
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
LLM	Large Language Model
OCC	Optimistic Concurrency Control
OS	Operating System
OT	Operational Transformation
PTY	Pseudo-Terminal
RAG	Retrieval-Augmented Generation
ReAct	Reason and Act Agent Paradigm
REST	Representational State Transfer

CHAPTER 1: INTRODUCTION

1.1 General

Modern software engineering has evolved into a highly distributed, data-intensive discipline where engineering velocity depends on seamless coordination across teams. Concurrently, the rapid maturation of generative artificial intelligence and Large Language Models (LLMs) has transformed code synthesis, automated debugging, and boilerplate generation. Despite these parallel advances, a fundamental structural disconnect persists in contemporary developer tooling: software development is inherently a collaborative team activity, whereas modern AI coding assistants are architected as isolated, single-user utilities. When an individual developer interacts with a conventional AI assistant, the resulting dialogue, architectural assumptions, and code mutations remain invisible to teammates until changes are manually committed to a version control repository.

Summit resolves this disconnect by embedding an autonomous, multi-turn AI coding agent directly into a browser-native, multiplayer Web IDE. Rather than treating the AI model as a private autocomplete buffer or an ephemeral chat sidebar, Summit elevates the agent to a shared, synchronous participant within a project room. Multiple developers connect simultaneously over full-duplex WebSockets to write code, execute terminal commands, inspect live web application previews, and collectively steer an autonomous agent whose memory of the repository's architecture persists across every session.

1.2 Background

Two major architectural transitions define the modern developer experience. The first transition is the migration from local desktop development environments to cloud-hosted, browser-accessible workspaces such as GitHub Codespaces and Replit, which enable engineering teams to share standardized execution environments without manual toolchain configuration. The second transition is the evolution of generative AI from single-line token completion to autonomous agentic workflows. Using the ReAct (Reason

and Act) paradigm, modern software agents iteratively inspect directory trees, read and modify source files, execute shell commands, and evaluate test suites.

However, existing cloud IDEs and agentic frameworks have developed along separate trajectories. Cloud IDEs support multi-user presence for human editors but relegate AI assistants to session-bound, single-user chat instances. Conversely, autonomous software engineering agents such as SWE-agent and Devin operate in non-interactive batch environments or isolated task containers, depriving human teams of real-time supervision, collaborative intervention, and shared institutional memory. Summit integrates these trajectories into a single coherent pipeline combining browser-based multi-user editing, real-time WebSocket telemetry broadcasting, ACID-compliant dual-tier relational memory, and optimistic concurrency enforcement.

1.3 Problem Statement

Contemporary developer environments and AI coding assistants suffer from two foundational architectural deficits that degrade team productivity and software quality. The first deficit is the **Episodic-Temporal Deficit**, wherein AI assistants operate ephemerally. When a developer initiates a new chat session or reloads the workspace, prior architectural choices, framework agreements, database schemas, and previously diagnosed bugs are erased from the active context. Consequently, developers are forced to repeatedly copy and paste redundant contextual specifications into prompts, inducing cognitive fatigue and resulting in hallucinated code that violates established team standards.

The second deficit is the **Single-User Isolation Paradigm**, wherein software teams collaborate on shared code repositories while AI assistants remain locked to single-user buffers. When Developer A instructs an AI assistant to refactor a backend API route, Developer B—working concurrently in the same repository—cannot observe the agent's real-time deliberation, tool execution, or terminal output. This lack of shared visibility leads to silent code overwrites, duplicated engineering labor, and fractured team alignment.

1.4 Motivation

The design of Summit is motivated by the operational requirement for a unified engineering workspace where human collaborators and autonomous agents share identical contextual ground. In team-based software development, an AI coding agent's actions affect the entire repository; therefore, the agent must operate as a shared, autonomous teammate whose live deliberation steps and tool invocations are visible to all collaborators in real time. Furthermore, a repository's architectural knowledge—such as database naming rules, authentication conventions, and module boundaries—represents durable institutional capital that should be automatically remembered across developer sessions without requiring repeated prompt engineering. Delivering code editing, terminal command execution, live web application previewing, and AI collaboration directly inside a standard web browser eliminates complex local toolchains and ensures consistent behavior across any client machine.

1.5 Objectives

The core engineering objectives of the Summit platform encompass five interconnected architectural goals. First, the project aims to design a browser-native collaborative IDE featuring Microsoft's Monaco Editor, an interactive Xterm.js terminal emulator, and a sandboxed live application preview. Second, it seeks to build a shared multiplayer AI agent that enables multiple concurrent developers to connect to a project room via WebSockets and jointly view, direct, pause, or resume an autonomous coding agent. Third, it implements a persistent dual-tier memory layer in an ACID-compliant SQLite database to retain append-only conversational history alongside structured architectural decisions and project conventions. Fourth, it develops a deterministic codebase context retrieval pipeline that scores and extracts relevant workspace files, memory records, and recent discussions within strict LLM token budgets. Fifth, it enforces Optimistic Concurrency Control (OCC) using versioned file states and visual side-by-side diff resolution to prevent destructive write collisions between concurrent developers and the autonomous agent.

1.6 Operational Boundaries and System Coverage

1.6.1 Included System Capabilities

Our project encompasses the end-to-end design, implementation, and functional verification of the full-stack Web IDE built with React 19, TypeScript, Vite, Monaco Editor, Xterm.js, and a Python FastAPI backend. The implementation includes an autonomous ReAct AI agent execution loop powered by LiteLLM, supporting dynamic routing across OpenAI, Anthropic, Gemini, DeepSeek, and local Ollama models, alongside an offline fallback generation engine. The agent is equipped with five core workspace tools (read_file, write_file, run_terminal, save_memory, and git_checkpoint) and broadcasts real-time telemetry across connected clients using 15 typed WebSocket event schemas. Persistence is handled through dual-tier relational storage in SQLite separating ephemeral conversation messages from persistent categorized project knowledge, complemented by file-level optimistic concurrency control returning HTTP 409 status codes on version mismatch and local Git checkpointing inside isolated project workspaces.

1.6.2 Operational Limitations

To maintain clear architectural boundaries during Phase I, concurrency control is enforced at the file level via OCC during save and tool-write operations, whereas character-by-character Operational Transformation (OT) or Conflict-Free Replicated Data Types (CRDTs) for simultaneous typing on the exact same line are reserved for Phase II. Similarly, terminal shell commands execute within path-validated local host subprocesses during Phase I, while hypervisor-isolated microVM or Docker container sandboxes are scheduled for Phase II production deployment. Finally, initial codebase context retrieval utilizes deterministic path-relevance keyword matching rather than dense vector embeddings, avoiding external vector database infrastructure overhead during Phase I validation.

1.7 Existing System

In the current software tooling landscape, collaborative editing, cloud execution, and autonomous AI code generation are divided across separate product categories. Desktop AI extensions such as GitHub Copilot and Cursor integrate into local VS Code installations, but their assistants are strictly single-tenant with memory bound to a local chat cache. Collaborative cloud IDEs such as Replit and GitHub Codespaces offer multiplayer editing or shared terminal environments, yet relegate AI capabilities to auxiliary single-user sidebars lacking multi-user agent presence. Autonomous software agents such as Devin and SWE-agent are capable of running terminal commands and file edits, but execute as black-box background tasks without interactive multiplayer rooms or cross-session relational memory. The primary capabilities and structural deficits of these existing systems are collected in Table 1.1.

Existing Approach	Primary Capability	Architectural Deficit
Desktop AI Extensions (GitHub Copilot, Cursor)	Inline code completion and local single-user chat inside desktop editors	Strictly single-tenant; lacks multiplayer agent visibility and cross-session relational memory
Collaborative Cloud IDEs (Replit, Codespaces)	Browser-accessible code editing, shared terminals, and hosted application previews	AI assistants operate as auxiliary single-user sidebars without shared team steering
Real-Time Co-Editors (VS Code Live Share)	Peer-to-peer cursor synchronization and shared debugging sessions	No native autonomous multi-tool agent or persistent repository memory store
Autonomous Software Agents (SWE-agent, Devin)	Multi-turn shell execution, repository navigation, and automated bug resolution	Executes as an opaque background task; lacks interactive multiplayer rooms and OCC diff resolution

Table 1.1: Limitations of existing developer environments and AI coding systems

1.8 Limitations of Existing System

The structural division across existing developer environments produces four critical operational limitations. First, existing tools suffer from **context amnesia**, where every session reset wipes out accumulated knowledge of previous architectural decisions, coding patterns, and debugging solutions. Second, they exhibit a **lack of multiplayer presence**, preventing teammates from observing live agent tool executions, deliberation traces, or terminal outputs initiated by a collaborator. Third, concurrent human and AI workflows are vulnerable to **write collisions**, where simultaneous edits between AI agents and human developers lead to unnotified file overwrites and lost work. Fourth, existing RAG solutions impose **high embedding and infrastructure overhead**, requiring external vector databases and complex indexing pipelines for basic workspace context assembly.

1.9 Proposed System

Summit unifies a collaborative browser-native Web IDE with an autonomous, multi-tenant AI coding agent backed by persistent relational memory. The FastAPI orchestration backend coordinates project sessions through full-duplex WebSocket channels, managing an asynchronous ReAct agent loop (SummitAgent). When any developer in a project room submits a prompt, SessionManager acquires the room execution lock, broadcasts the agent's active status to all connected peers, and invokes ContextBuilder.

ContextBuilder queries the SQLite database (summit.db) to retrieve deduplicated project memories (project_memories), recent multi-user dialogue (messages), and path-scored workspace files. The assembled prompt is dispatched through LiteLLM to the configured model endpoint. As the agent deliberates and invokes tools—reading files, writing code, running terminal commands, saving new architectural memories, or creating Git checkpoints—every step is serialized into a typed WebSocket envelope and streamed to all connected browsers in under 50 milliseconds. File mutations are guarded by WorkspaceManager using Optimistic Concurrency Control: if a human developer or the

agent attempts to write a file using a stale version number, the server rejects the write with an HTTP 409 Conflict and triggers an interactive side-by-side Monaco diff resolution modal on the client.

1.10 Advantages of Proposed System

Summit provides five major engineering advantages over existing single-user or stateless tools. First, it preserves **persistent institutional knowledge**, ensuring that architectural guidelines and project decisions persist across sessions in SQLite and are automatically injected into future agent prompts. Second, it establishes **real-time team alignment**, as all developers in a project room simultaneously observe the agent's deliberation steps, terminal commands, and file edits as they happen. Third, it enforces **conflict-resilient collaboration**, using version-checked file saves to prevent the agent or teammates from accidentally overwriting concurrent code edits. Fourth, it requires **zero local setup**, operating as a fully browser-native development environment with integrated Monaco code editing, shell access, and live application previewing. Fifth, it delivers **model flexibility**, using agnostic LiteLLM routing to switch seamlessly between commercial cloud providers and local open-source models without altering workspace logic.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

The architectural design of Summit sits at the intersection of Computer-Supported Cooperative Work (CSCW), Human-Computer Interaction (HCI), repository-level Retrieval-Augmented Generation (RAG), and autonomous software engineering agents. This chapter reviews nine foundational studies arranged chronologically from classical concurrency control in groupware to modern empirical evaluations of AI pair programming, repository-level prompt synthesis, and Agent-Computer Interfaces (ACI). Examining these works together reveals how individual components—collaborative editing, context retrieval, and agentic tool use—have matured independently while leaving open the architectural synthesis of a multiplayer, memory-persistent agentic IDE.

2.2 Review of Related Work

[1] Ellis and Gibbs (1989)

Ellis and Gibbs established the theoretical foundation for real-time collaborative groupware by introducing Operational Transformation (OT) for concurrency control in distributed document editors. Their work demonstrated that concurrent users editing a shared state without locking require deterministic transformation or version reconciliation to preserve state convergence. In the context of software engineering workspaces where an autonomous AI agent performs wholesale multi-line file replacements alongside human editors, Summit adapts this concurrency principle at the file boundary through Optimistic Concurrency Control (OCC) with monotonic integer versioning (V \in \mathbb{N}^+) and visual diff arbitration.

[2] Oster, Urso, Molli, and Imine (2006)

Oster et al. advanced collaborative editing theory by formulating Conflict-Free Replicated Data Types (CRDTs) for optimistic replication without central transformation bottlenecks. Their study proved that commutative concurrent operations allow

decentralized peers to converge automatically, while highlighting the complexity of maintaining syntactic validity when automated batch mutations intersect with human keystrokes. This finding informs Summit's phased concurrency architecture: Phase I enforces lightweight, ACID-backed file-level OCC to guard against agent write collisions immediately, while establishing the clean WebSocket room abstraction required to layer Yjs CRDT keystroke synchronization in Phase II.

[3] Peng, Kalliamvakou, Cihon, and Demirer (2023)

Peng et al. conducted a landmark randomized controlled trial measuring the productivity impact of GitHub Copilot on 95 recruited software developers tasked with implementing an HTTP server in JavaScript. Developers in the treatment group with access to the AI pair programmer completed the task in an average of 71.17 minutes compared to 160.89 minutes for the control group—a statistically significant 55.8% reduction in completion time ($p = 0.0017$, 95% CI [21\%, 89\%]). Notably, heterogeneous treatment analysis revealed that less-experienced developers and those coding more hours daily derived substantial velocity gains. Summit builds directly upon this empirical validation of AI-assisted coding, while removing the single-user isolation constraint of Copilot so that entire teams—including junior engineers—can observe and learn from shared agent interactions simultaneously.

[4] Ross, Martinez, Houde, Muller, and Weisz (2023)

Ross et al. investigated conversational interaction with Large Language Models for software development through "The Programmer's Assistant," evaluating how developers engage in multi-turn dialogue beyond single-shot code completion. Their empirical observations showed that conversational follow-ups significantly improve code comprehension and iterative refinement, but exposed a critical usability friction: developers had to manually supply surrounding code context and re-explain project constraints whenever conversational sessions reset. Summit directly addresses the contextual onboarding friction identified by Ross et al. through its automated

ContextBuilder and persistent SQLite project_memories table.

[5] Shrivastava, Larochelle, and Tarlow (2023)

Shrivastava et al. proposed the Repo-Level Prompt Generator (RLPG) at ICML 2023 to resolve the lack of repository-wide awareness in code LLMs. By extracting context from ten distinct repository sources—including current files, parent classes, imported dependencies, and sibling directory files—and selecting optimal prompt proposals, RLPG achieved up to a 17% relative improvement over standard single-file Codex baselines. Their findings confirm that injecting structural repository context and cross-file cues into LLM prompts substantially reduces unresolved symbol and import hallucinations. Summit adapts this finding via a lightweight, deterministic path-relevance scoring engine that ranks workspace files against user prompt tokens without requiring GPU-heavy prompt proposal training.

2.3 Comparative Analysis

Table 2.1 synthesizes the methodology, study focus, and core architectural takeaways of the reviewed literature.

Ref.	**Author(s) & Year**	**Study Focus**	**Methodology / Approach**	**Reported Architectural Takeaway**
[1]	Ellis & Gibbs (1989)	Real-time groupware concurrency	Operational Transformation (OT) on concurrent document state mutations	Concurrent multi-actor editing requires formal version verification to prevent destructive

Ref.	Author(s) & Year	Study Focus	Methodology / Approach	Reported Architectural Takeaway
				overwrites
[2]	Oster et al. (2006)	Optimistic replication in collaborative editors	Conflict-Free Replicated Data Types (CRDTs) for commutative text synchronization	Decentralised character edits converge cleanly, whereas batch file mutations benefit from file-level version gates
[3]	Peng et al. (2023)	Developer productivity with AI pair programming	Controlled trial (N=95) implementing an HTTP server with GitHub Copilot	AI assistance reduces task completion time by 55.8% (71.17 min vs 160.89 min, p = 0.0017)
[4]	Ross et al. (2023)	Conversational LLM interaction for coding	Empirical user evaluation of multi-turn dialogue in "The Programmer's Assistant"	Multi-turn chat aids comprehension, but stateless resets force repetitive manual context onboarding
[5]	Shrivastava et al. (2023)	Repository-level prompt	Learned prompt proposal	Injecting cross-file

Ref.	Author(s) & Year	Study Focus	Methodology / Approach	Reported Architectural Takeaway
		generation (RLPG)	classifier across 10 repository context sources	repository structure yields up to 17% relative improvement over single-file baselines

Table 2.1: Literature survey comparison across collaborative editing, code retrieval, and agentic systems

To ground the architectural decisions of Summit in quantitative evidence from the literature, the visual below illustrates both the developer velocity gains of AI pair programming reported by Peng et al. and the repository-level completion accuracy gains reported by Zhang et al..

As the empirical data demonstrates, providing developers with an AI pair programmer cuts implementation time by more than half, while expanding the model's prompt beyond the single open buffer to include repository-wide files sharing path and import structures increases API invocation exact match accuracy by more than 15 percentage points. Summit synthesizes both findings: it exposes the AI agent to the entire team in a shared workspace while automatically assembling path-scored repository files and persistent architectural memories into every prompt.

2.4 Comparative Analysis of the Proposed System

To position Summit precisely within the current ecosystem, Table 2.2 contrasts nine core collaboration, execution, and memory dimensions across GitHub Copilot, SWE-agent, Devin / OpenDevin, VS Code Live Share, Replit AI, and Summit.

Capability / Feature	GitHub Copilot	SWE-agent	Devin / OpenDevin	VS Code Live Share	Replit AI	**Summit (Proposed)**
Browser-Native Web IDE	No	No	Partial (Viewer)	No	Yes	**Yes (Monaco + Xterm)**
Multiplayer Agent Room	No	No	No	Yes (Human only)	Partial (Separate)	**Yes (Shared Room)**
Real-Time Telemetry Broadcast	No	No	No	No	No	**Yes (15 Event Types)**
Cross-Session Project Memory	No	No	No	No	Partial	**Yes (SQLite Relational)**
Concurrency Control	No	No	No	Partial (P2P)	Partial (Lock)	**Yes (OCC HTTP 409)**
Autonomous Multi-Tool Agent	No	Yes	Yes	No	Partial	**Yes (5 Core Tools)**
Integrated Terminal (Xterm.js)	No	No	Partial (Logs)	Partial	Yes	**Yes (Bidirectional)**
Live Sandboxed Web Preview	No	No	Partial	Partial	Yes	**Yes (Dynamic Iframe)**
Multi-LLM	No	No	Partial	No	No	**Yes**

Capability / Feature	GitHub Copilot	SWE-agent	Devin / OpenDevin	VS Code Live Share	Replit AI	**Summit (Proposed)**
Provider Routing						**(LiteLLM)**

Table 2.2: Comparative analysis of state-of-the-art developer systems versus Summit

Compared against existing platforms, Summit establishes three decisive architectural distinctions. Versus desktop extensions such as GitHub Copilot and Cursor, Summit eliminates single-developer buffer isolation by enabling real-time multi-user shared prompts and transparent agent action streams. Versus autonomous batch agents such as SWE-agent and Devin, Summit provides an interactive web workspace equipped with active human steering, lifecycle controls (PAUSE, RESUME, STOP), and side-by-side diff inspection. Versus collaborative cloud workspaces such as Replit and VS Code Live Share, Summit treats the AI agent as a first-class collaborative room participant whose file mutations are subject to Optimistic Concurrency Control verification and whose architectural discoveries are recorded to permanent relational project memory.

2.5 Research Gap

A synthesis of the literature reveals a clear structural dichotomy: prior systems address either **real-time human-to-human collaboration** (via OT/CRDT editors and peer-to-peer screen sharing) or **autonomous single-developer agent execution** (via batch Agent-Computer Interfaces). No existing platform bridges the intersection of synchronous multi-user presence in a web development environment, shared live steering of an autonomous multi-tool coding agent, and cross-session relational project memory that permanently eliminates context loss. The contribution of Summit is therefore architectural and integrative: it binds multiplayer WebSocket coordination, deterministic repository context assembly, dual-tier relational memory, and file-level optimistic concurrency control into a unified full-stack platform.

CHAPTER 3: THEORETICAL BACKGROUND AND SYSTEM DESIGN

3.1 General

Summit is engineered as a modular, asynchronous four-tier platform comprising a browser-native single-page application (SPA), a FastAPI REST and WebSocket orchestration gateway, a LiteLLM-routed ReAct inference engine, and an ACID-compliant SQLite and sandboxed filesystem persistence tier. Separating client presentation, room synchronization, context assembly, and storage ensures that individual subsystems—such as the underlying LLM provider or the concurrency validator—can be upgraded without altering upstream or downstream interfaces.

3.2 Browser-Native IDE Rendering and Monaco Editor Core

The client editing surface is built on React 19 and TypeScript, embedding Microsoft's Monaco Editor (@monaco-editor/react), the foundational editing engine that powers Visual Studio Code. Monaco provides an in-browser Abstract Syntax Tree (AST) tokenizer, multi-language syntax highlighting, inline diagnostic linting, and a native side-by-side diff editor. Each open file buffer in the browser maintains both its active text state and its last synchronized monotonic integer version (V_{\text{expected}}), enabling deterministic conflict checks whenever a developer triggers a save or receives a broadcast FILE_CHANGED event from the server.

3.3 Asynchronous Bidirectional Terminal Emulation

To allow developers and the autonomous agent to compile code, install packages, and run test suites directly from the browser, Summit integrates Xterm.js (@xterm/xterm) on the frontend paired with an asynchronous subprocess runner (TerminalService) on the backend. Xterm.js renders a hardware-accelerated HTML5 canvas supporting full ANSI escape sequences, color codes, and clickable file links. Commands entered by a

developer or invoked by the agent's run_terminal tool are executed asynchronously inside the project's isolated directory, streaming standard output (stdout) and standard error (stderr) chunks across the room's WebSocket channel via TERMINAL_OUTPUT events without blocking the main Python asyncio event loop.

3.4 Sandboxed Live Web Application Previewing

Modern web development requires immediate visual feedback alongside code modifications. Summit incorporates a dynamic, sandboxed HTML5 <iframe> preview subsystem backed by the /api/projects/{id}/preview router. When the agent or a human developer modifies HTML, CSS, or JavaScript assets and saves the buffer, the client automatically refreshes the sandboxed preview frame, rendering the live application within restricted browser security boundaries (sandbox="allow-scripts allow-same-origin") to prevent cross-frame script leakage.

3.5 Full-Duplex WebSocket Communication Protocol

Real-time multiplayer collaboration requires sub-50ms event propagation across all connected peers. Summit utilizes the RFC 6455 WebSocket protocol via FastAPI's native ASGI WebSocket implementation at the /api/projects/{id}/agent/stream endpoint. Unlike half-duplex HTTP polling, RFC 6455 maintains a persistent, full-duplex TCP connection between each browser client and the SessionManager, enabling simultaneous upstream transmission of user prompts and lifecycle signals (PAUSE, RESUME, STOP) alongside downstream broadcasting of 15 strongly typed telemetry event schemas.

3.6 Multiplayer Room Synchronization and Presence State Management

Within the orchestration tier, SessionManager acts as the centralized room hub. For each active project_id, SessionManager maintains an in-memory roster of connected WebSocket clients, user metadata (display name, assigned avatar color), and an asynchronous room execution lock (asyncio.Lock). When a developer connects or disconnects, SessionManager broadcasts USER_JOINED or USER_LEFT envelopes to

update the live avatar presence bar in every peer's browser. The room lock guarantees that only one autonomous agent execution loop runs per project room at any instant, queuing or rejecting overlapping prompts while allowing any participant in the room to observe or pause the active agent run.

3.7 ReAct (Reason and Act) Autonomous Agent Execution Pattern

The autonomous coding agent (SummitAgent) implements the multi-turn ReAct (Reason and Act) execution paradigm. Rather than attempting to generate an entire multi-file software change in a single unverified pass, the agent iterates through a closed feedback loop consisting of three states. During the deliberation phase (THINKING), the model evaluates the user prompt, repository context, and prior tool outputs to formulate an execution plan, streaming AGENT_THOUGHT tokens to all participants in the room. During the tool invocation phase (EXECUTING), the model emits structured JSON tool calls (read_file, write_file, run_terminal, save_memory, or git_checkpoint), triggering TOOL_START events across the WebSocket channel. Finally, during the observation evaluation phase, SummitAgent captures the return payload or error trace from the workspace, emits a TOOL_END event, appends the observation to the active message array, and re-queries the model until the task is verified complete or the maximum turn ceiling is reached.

3.8 Unified Multi-Provider LLM Routing via LiteLLM

To prevent vendor lock-in and support heterogeneous deployment budgets, Summit decouples the ReAct loop from proprietary model SDKs by routing all inference calls through LiteLLM. LiteLLM translates OpenAI-compatible message and tool-calling schemas into the native API contracts of Anthropic (Claude 3.5 Sonnet), OpenAI (GPT-4o, GPT-4o-mini), Google (Gemini 2.5 Pro), DeepSeek (DeepSeek-V3), and local self-hosted Ollama models. Furthermore, Summit incorporates a deterministic offline fallback generation engine so that workspace mechanics, WebSocket streaming, and automated test suites remain fully verifiable even in air-gapped environments without

external API keys.

3.9 Agent Tool Invocation Interface and Workspace Sandboxing

The Agent-Computer Interface exposes five foundational tools to the LLM. The read_file(path) tool reads the UTF-8 content and current integer version of a target file inside the workspace. The write_file(path, content) tool creates or updates a file, incrementing its version in file_versions and broadcasting a FILE_CHANGED event to all connected browsers. The run_terminal(command) tool executes a shell command inside the project root and streams TERMINAL_OUTPUT back to both the room and the agent context. The save_memory(category, key, value) tool persists a structured architectural rule, convention, decision, or task status to SQLite via upsert semantics and emits a MEMORY_UPDATED notification. The git_checkpoint(message) tool stages workspace modifications, creates a Git commit, records the hash in git_checkpoints, and broadcasts a checkpoint notification. Every file and shell operation is strictly validated by WorkspaceManager using canonical path resolution (os.path.realpath), ensuring that any attempt to resolve a path outside the designated project_dir (such as ../../etc/passwd or ../secret.txt) immediately raises a ValueError and preserves strict workspace isolation.

3.10 Dual-Tier Relational Memory Architecture

To resolve the episodic-temporal deficit, Summit formalizes a Dual-Tier Relational Memory Model stored in SQLite (summit.db). The architecture decouples sequential conversational history (\mathcal{M}_{\text{conv}}) from structured, deduplicated institutional knowledge (\mathcal{M}_{\text{proj}}):

\mathcal{M}_{\text{conv}} = \langle (u_1, r_1, c_1, t_1), (u_2, r_2, c_2, t_2), \dots, (u_n, r_n, c_n, t_n) \rangle

where u_i denotes the originating user_id, r_i \in \{\text{user}, \text{assistant}, \text{system}, \text{tool}\} denotes the role, c_i represents the message content, and t_i is the monotonic timestamp in the messages table. Conversely, the persistent project memory tier is defined as a relational set of categorized key-value tuples:

\mathcal{M}_{\text{proj}} = \{ (category, key, value) \mid category \in

\{\text{architecture}, \text{convention}, \text{decision}, \text{task}\} \}
When the agent invokes save_memory or when MemoryExtractor parses architectural declarations, records are written to the project_memories table using atomic SQLite upsert semantics (INSERT INTO project_memories ... ON CONFLICT(project_id, key) DO UPDATE SET category = excluded.category, value = excluded.value, updated_at = CURRENT_TIMESTAMP). This relational constraint guarantees that evolving project decisions overwrite stale entries for the same key in place without creating duplicate rows or bloating the prompt token budget.

3.11 Deterministic Path-Relevance Context Retrieval and Prompt Assembly

Before dispatching an inference request to LiteLLM, ContextBuilder dynamically assembles the system prompt and workspace context within a strict token budget through a three-stage deterministic assembly pipeline. In the first stage, persistent memory injection retrieves all active records from \mathcal{M}_{\text{proj}} for the project room, organized across architecture, convention, decision, and task categories. In the second stage, path-relevance file scoring scans the project workspace tree and computes a deterministic relevance score S(f, Q) for each file path f against the tokenized user prompt Q:

S(f, Q) = w_{\text{exact}} \cdot \mathbb{I}(\text{basename}(f) \in Q) + w_{\text{path}} \cdot | \text{tokens}(f) \cap \text{tokens}(Q) | + w_{\text{recency}} \cdot \text{RecencyRank}(f)

Files achieving the highest path-relevance scores have their contents injected up to the maximum context window allocation. In the third stage, a sliding conversational window appends the most recent multi-user turns from \mathcal{M}_{\text{conv}} (messages), preserving immediate dialogue continuity without overflowing the model's context limit.

3.12 Optimistic Concurrency Control (OCC) and Monotonic Versioning

To prevent write collisions when multiple developers and the AI agent edit files concurrently, WorkspaceManager enforces Optimistic Concurrency Control (OCC). Every file in a project workspace is assigned a monotonically increasing integer version

V \in \mathbb{N}^+ tracked in the file_versions table. When a client or agent submits a write request for file_id with new content C_{\text{new}} and expected version V_{\text{expected}}, the write succeeds if and only if the expected version matches the active server version V_{\text{current}}:

\text{Write}(file_id, C_{\text{new}}, V_{\text{expected}}) \iff V_{\text{expected}} == V_{\text{current}}

Upon a valid match, the server atomically writes C_{\text{new}} to disk, increments the version to V_{\text{current}} + 1, records the snapshot in file_versions, and broadcasts a FILE_CHANGED event containing the new version number. If V_{\text{expected}} \neq V_{\text{current}}, the transaction is aborted and the FastAPI router immediately returns an **HTTP 409 Conflict** response carrying the server's current content C_{\text{current}} and active version V_{\text{current}}.

3.13 Visual Conflict Resolution and Side-by-Side Diff Inspection

When a browser client receives an HTTP 409 Conflict response during a file save, Summit intercepts the error and launches the Visual Conflict Resolution Modal. This component instantiates Monaco’s native DiffEditor, rendering the server's latest version (C_{\text{current}}, potentially modified by the AI agent or a teammate) on the left pane and the developer's local unsaved buffer (C_{\text{local}}) on the right pane with line-by-line additions and deletions highlighted. The developer can inspect the exact divergence and choose to accept the server state, force-overwrite with the local state (updating V_{\text{expected}} \leftarrow V_{\text{current}}), or manually merge lines before committing.

3.14 Automated Git Checkpointing and Rollback Mechanics

Alongside SQLite file versioning, Summit integrates native Git repository management through GitService. Every project workspace is initialized as an isolated local Git repository. Whenever the autonomous agent completes a major multi-file milestone, it invokes the git_checkpoint tool, which stages modified files (git add -A), commits them with a descriptive semantic message, and logs the resulting SHA-1 commit hash in the

git_checkpoints table. Developers can also query /api/projects/{id}/git to inspect working tree diffs or roll the entire workspace back to a verified checkpoint if an experimental agent refactoring proves undesirable.

3.15 Relational Schema Design and ACID Transaction Guarantees

Summit utilizes SQLite3 accessed via SQLAlchemy and aiosqlite to guarantee Atomicity, Consistency, Isolation, and Durability (ACID) across all state mutations. By enabling Write-Ahead Logging (PRAGMA journal_mode=WAL;), SQLite supports concurrent non-blocking reads from ContextBuilder while serializing transactional writes from SessionManager, MemoryExtractor, and WorkspaceManager without requiring an external database server daemon.

3.16 FastAPI Asynchronous Orchestration Layer

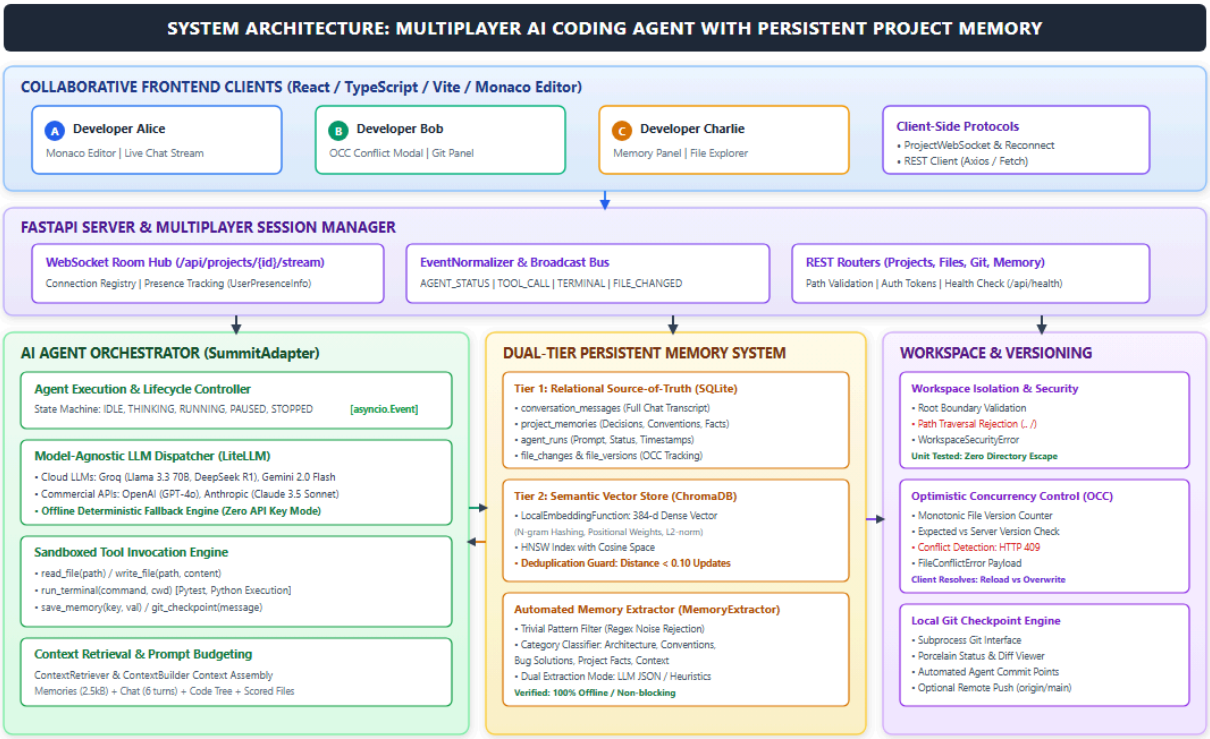
The backend API gateway is implemented in Python 3.11+ using FastAPI. FastAPI's asynchronous ASGI foundation allows a single server process to multiplex dozens of persistent WebSocket connections, REST file operations, and non-blocking asyncio.create_subprocess_shell terminal executions concurrently. All incoming REST payloads and outgoing WebSocket frames are validated through strict Pydantic v2 models, ensuring contract integrity between the TypeScript frontend and Python backend.

3.17 System Architecture

The overall system architecture of Summit is organized into four distinct horizontal tiers connected by explicit, type-checked communication conduits. Tier 1 (Client Tier: Browser Web IDE) hosts two primary subsystem groups: the Editor Modules—comprising the Monaco Editor (handling syntax highlighting, OCC version tracking, and diffs), the Xterm.js Subsystem (providing terminal PTY emulation), and the Sandboxed Web Preview Iframe—and the Collaboration Modules, which encompass the Presence Tracker (displaying live user avatars), the Agent Stream Panel and Chat Feed, and the Visual Conflict Resolution Modal.

Tier 1 communicates over full-duplex WebSockets (for real-time telemetry events) and HTTP REST with Tier 2 (Orchestration & API Tier implemented in FastAPI). Tier 2 exposes six modular API routers (/projects, /files, /agent/stream, /terminal, /preview, and /git) backed by two internal service clusters: Core Services (SessionManager for rooms and locks, WorkspaceManager for files and OCC, and GitService & Subprocess Runner) and the AI Context Subsystem (ContextBuilder & Prompt Assembler, the Path-Relevance Scoring Engine, and MemoryExtractor & Deduplicator).

Tier 2 dispatches tool calls and receives streaming token completions from Tier 3 (AI Inference Engine), which executes the ReAct Multi-Turn Autonomous Agent Loop via the LiteLLM Multi-Provider Dispatcher across supported cloud endpoints (Claude 3.5 Sonnet, GPT-4o, DeepSeek-V3, Gemini 2.5 Pro), local self-hosted Ollama models, and the Offline Fallback Generation Engine. In parallel, Tier 2 executes SQL queries and file-level OCC checks against Tier 4 (Storage & Persistence Tier), which houses the relational SQLite database (summit.db containing users, projects, messages, project_memories, file_versions, and git_checkpoints) alongside the Sandboxed Workspace Filesystem. Table 3.1 provides the complete structural specification of the four architectural tiers and their inter-tier protocols.



Architectural Tier	Subsystem Group	Constituent Modules & Endpoints	Inter-Tier Communication Protocol
1. Client Tier (Browser Web IDE)	Editor Modules & Collaboration Modules	Monaco Editor (Syntax, OCC, Diffs), Xterm.js Subsystem (Terminal PTY), Sandboxed Web Preview Iframe, Presence Tracker (User Avatars), Agent Stream Panel & Chat Feed, Visual Conflict Resolution Modal	Bidirectional RFC 6455 WebSockets (15 real-time telemetry events) and HTTP REST to Tier 2
2. Orchestration & API Tier (FastAPI)	API Routers, Core Services & AI	Routers: /projects, /files, /agent/stream,	Upstream WebSockets/REST to

Architectural Tier	Subsystem Group	Constituent Modules & Endpoints	Inter-Tier Communication Protocol
	Context Subsystem	/terminal, /preview, /git; Core Services: SessionManager, WorkspaceManager, GitService & Subprocess Runner; AI Context: ContextBuilder, Path-Relevance Scorer, MemoryExtractor	Tier 1; Tool Dispatch & Streaming to Tier 3; SQL Queries & File OCC to Tier 4
3. AI Inference Engine	Autonomous ReAct & Multi-Provider Routing	ReAct Multi-Turn Autonomous Agent Loop, LiteLLM Multi-Provider Dispatcher, Cloud Models (Claude 3.5 Sonnet, GPT-4o, DeepSeek-V3, Gemini 2.5 Pro), Local Models (Ollama), Offline Fallback Engine	Asynchronous JSON tool-call dispatch and token streaming with Tier 2
4. Storage & Persistence Tier	Relational Database & Workspace Sandbox	SQLite Database (summit.db: users, projects, messages,	Transactional aiosqlite SQL queries, ACID upserts, and

Architectural Tier	Subsystem Group	Constituent Modules & Endpoints	Inter-Tier Communication Protocol
		project_memories, file_versions, git_checkpoints) and Sandboxed Workspace Filesystem	path-sanitized POSIX/Windows file I/O with Tier 2

Table 3.1: Four-tier architectural specification and inter-tier communication protocols

3.18 Data Flow Diagram

The Data Flow Diagram (DFD) models the movement and transformation of information through Summit across three balanced hierarchical levels. External entities, processing nodes, and persistent data stores are strictly balanced so that every input and output crossing the boundary of a parent process reappears unchanged in its child decomposition.

3.18.1 Level 0 (Context Diagram)

At Level 0, the entire Summit platform is represented as a single central process (0.0 SUMMIT SYSTEM) interacting with two external boundaries: concurrent human **Developers (Users 1 \dots N)** and the underlying **SQLite Database & Workspace Files** store. Developers supply multiplayer prompts, code edits, lifecycle signals, and terminal inputs into Process 0.0, and receive real-time broadcast telemetry, OCC file diffs, presence updates, and terminal output streams in return, while Process 0.0 reads and writes persistent state to the SQLite database and workspace files.

3.18.2 Level 1

Level 1 decomposes Process 0.0 into five sequential and cyclic operational processes.

Process 1.0 (SessionManager) receives incoming user prompts and lifecycle controls from Developers, validates the project room execution lock, updates room presence rosters, and broadcasts AGENT_STATUS changes. **Process 2.0 (ContextBuilder)** queries persistent architectural records from project_memories (Store D1), recent dialogue turns from messages (Store D2), and the workspace file tree (Store D3) to assemble a token-budgeted prompt payload. **Process 3.0 (LiteLLM Router)** dispatches the assembled prompt and tool schemas to the configured cloud or local AI model endpoint and streams raw model completions and structured tool invocations. **Process 4.0 (Tool Execution Loop)** parses and executes agent tool calls—reading and writing versioned files, executing shell subprocesses, saving project memories, and recording Git checkpoints. Finally, **Process 5.0 (Broadcast Engine)** normalizes all tool outputs, deliberation tokens, file version updates, and terminal streams into 15 standardized WebSocket event envelopes and streams them to all connected WebSocket clients in the project room.

3.18.3 Level 2

Level 2 decomposes Process 4.0 (Tool Execution Loop) into three granular sub-transformations. **Process 4.1 (Path Sanitizer & OCC Validator)** resolves candidate file paths against the sandbox root to block directory traversal and verifies V_{\text{expected}} == V_{\text{current}} against Store D3 (file_versions) before committing any file mutation. **Process 4.2 (Asynchronous Subprocess Runner)** spawns non-blocking shell processes, capturing stdout and stderr pipes and forwarding chunks to Process 5.0. **Process 4.3 (Memory Deduplicator & Git Checkpointer)** executes ON

DFD Level	Process Node	Source Entity / Store	Input Data Flow	Target Entity / Store	Output Data Flow
Level 0	0.0 Summit System	Developers (Users 1 \dots N)	Multiplayer prompts, file edits, terminal inputs	Developers & SQLite / Workspace	Broadcast telemetry, OCC diffs, terminal

DFD Level	Process Node	Source Entity / Store	Input Data Flow	Target Entity / Store	Output Data Flow
					streams, persisted records
Level 1	1.0 SessionManager	Developers (Users 1 \dots N)	User prompt & lifecycle controls (PAUSE, RESUME, STOP)	2.0 ContextBuilder & Room Peers	Validated room lock acquisition & AGENT_STATUS broadcast
Level 1	2.0 ContextBuilder	1.0 SessionManager & Stores D1–D3	Active prompt, project_memories, messages, file tree	3.0 LiteLLM Router	Assembled token-budgeted system & workspace prompt payload
Level 1	3.0 LiteLLM Router	2.0 ContextBuilder	Prompt payload & 5 JSON tool schemas	4.0 Tool Execution Loop	Streamed thought tokens (AGENT_THOUGHT) & structured tool calls
Level 1	4.0 Tool Execution Loop	3.0 LiteLLM Router	Tool invocation arguments (read_file, write_file,	Stores D1–D4 & 5.0 Broadcast Engine	File writes, shell execution streams, memory

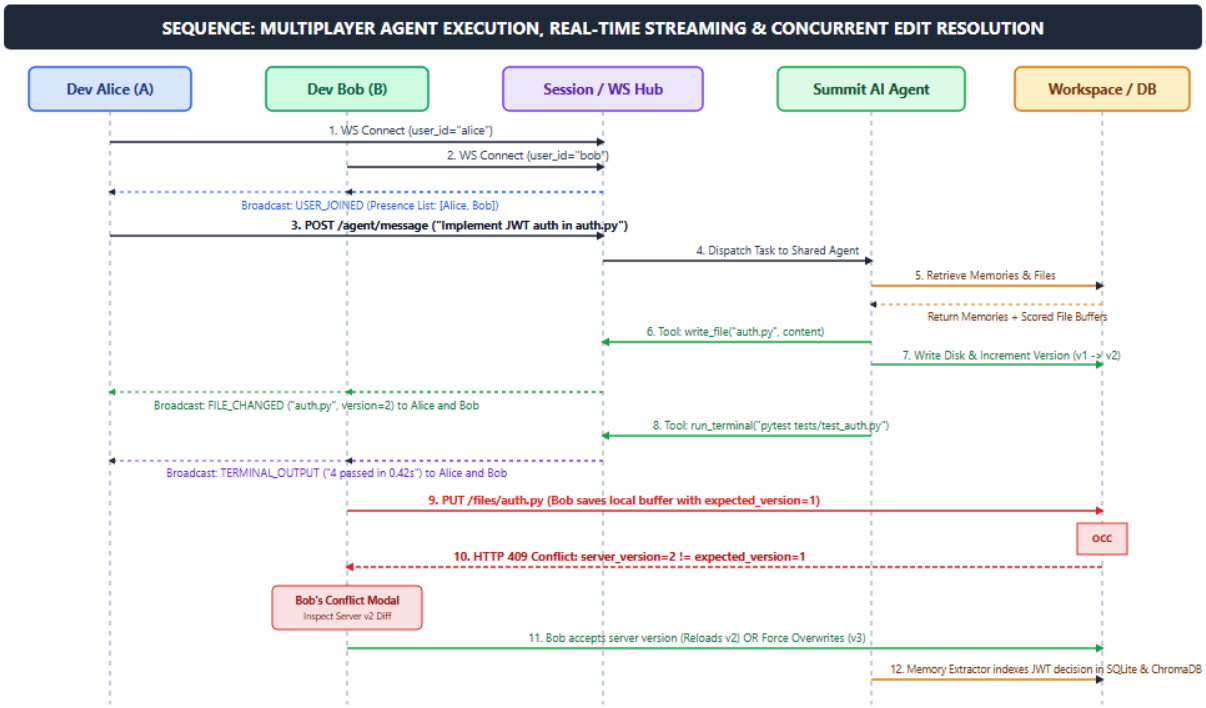
DFD Level	Process Node	Source Entity / Store	Input Data Flow	Target Entity / Store	Output Data Flow
			etc.)		upserts, Git commits
Level 1	5.0 Broadcast Engine	4.0 Tool Execution Loop	Normalized tool observations, file diffs, terminal chunks	Connected WebSocket Clients	15 typed WebSocket event envelopes streamed in <50\text{ ms}
Level 2	4.1 Path & OCC Validator	4.0 Tool Execution Loop	Candidate path, C_{\text{new}}, V_{\text{expected}}	Store D3 (file_versions)	Versioned file write (V+1) or HTTP 409 Conflict / ValueError
Level 2	4.2 Subprocess Runner	4.0 Tool Execution Loop	Shell command string (run_terminal)	5.0 Broadcast Engine	Non-blocking stdout/stderr chunks (TERMINAL_OUTPUT) & exit code
Level 2	4.3 Memory & Git Engine	4.0 Tool Execution Loop	Memory tuple (category, key, value) or checkpoint message	Stores D1 (project_memories) & D4	Atomic SQL upsert (MEMORY_UPDATED) & Git SHA-1 commit record

Table 3.2: Hierarchical Data Flow Diagram (Level 0, Level 1, and Level 2) process and store mapping

3.19 Sequence Diagram

The operational activity workflow of a collaborative session in Summit proceeds through four synchronized phases. During the initial **Room Initialization and Synchronization** phase, when a developer opens a project workspace in the browser, the client fetches the current file tree, recent messages, and persistent memories via REST, while opening a persistent WebSocket connection to /api/projects/{id}/agent/stream, where SessionManager registers the socket and broadcasts USER_JOINED to existing peers. During the second **Concurrent Human and Agent Activity Fork** phase, execution branches into parallel interactive tracks: on the human track, developers edit code in Monaco, run commands in Xterm.js, and preview web outputs, while on the agent track, a submitted prompt acquires the room lock, transitions the room state to THINKING, and invokes ContextBuilder to assemble memories, messages, and path-scored files.

During the third **Iterative ReAct Tool Loop and Lifecycle Check** phase, SummitAgent inspects the room lifecycle state before each turn; if a teammate has triggered PAUSE, the loop suspends cleanly until RESUME is received, whereas if STOP is triggered, the loop aborts and releases the room lock. Otherwise, the agent invokes LiteLLM, executes any requested tools (read_file, write_file, run_terminal, save_memory, git_checkpoint), and broadcasts real-time telemetry events (TOOL_START, TOOL_END, FILE_CHANGED, MEMORY_UPDATED) to all connected browsers. Finally, during the **Concurrency Arbitration and Convergence** phase, whenever a human save or agent write occurs, WorkspaceManager compares V_{\text{expected}} against V_{\text{current}}; valid writes increment the version to V_{\text{current}} + 1 and notify the room, whereas mismatched versions return HTTP 409 Conflict, routing the client to the side-by-side diff modal until versions converge and the room returns to IDLE.



3.20 Overall System Diagram

The overall system topology unites the multi-user client workspace, the asynchronous WebSocket session hub, the ReAct agent loop, and the dual-tier persistence engine into a single closed-loop control surface. Regardless of whether an action originates from Developer A typing in Monaco, Developer B running a test suite in Xterm.js, or SummitAgent refactoring a module via write_file, all workspace mutations pass through WorkspaceManager and SessionManager. Consequently, every participant in the room observes a consistent, real-time view of the codebase, terminal, and institutional memory.

3.21 Core Conceptual Classes and Database Schema

The object-oriented and service-oriented design of the Summit backend is encapsulated in six core conceptual classes. Table 3.3 defines the representative attributes and methods of each class.

Class / Service Name	Representative Attributes	Representative Methods
SessionManager	active_rooms: Dict[str,	connect_user(),

Class / Service Name	Representative Attributes	Representative Methods
	List[WebSocket]], room_locks: Dict[str, asyncio.Lock], presence_roster: Dict[str, UserPresence]	disconnect_user(), broadcast_event(), acquire_agent_lock(), set_agent_lifecycle_state()
WorkspaceManager	workspace_root: Path, db_session: AsyncSession	validate_safe_path(), read_file(), write_file_with_occ(), get_file_tree(), record_file_version()
ContextBuilder	max_token_budget: int, path_weights: Dict[str, float]	build_agent_prompt(), score_file_relevance(), fetch_project_memories(), fetch_recent_messages()
SummitAgent	model_name: str, max_turns: int, lifecycle_state: AgentStatus, tools: List[ToolSchema]	run_react_loop(), dispatch_tool_call(), handle_pause_resume(), fallback_offline_response()
MemoryExtractor	valid_categories: Set[str], db_session: AsyncSession	upsert_memory(), extract_memories_from_turn(), deduplicate_keys(), get_categorized_memories()
GitService & TerminalService	project_dir: Path, active_subprocesses: Dict[str, Process]	init_repo(), create_checkpoint(), get_diff(), execute_command_stream()

Table 3.3: Core conceptual classes and service abstractions

The persistent state of the platform is maintained inside the SQLite relational database (summit.db) across six normalized tables. Table 3.4 documents the schema definitions, primary keys, and integrity constraints of each table.

Table Name	Column Schema & Data Types	Integrity & Indexing Constraints	Architectural Purpose
projects	id TEXT, name TEXT, created_at TIMESTAMP	PRIMARY KEY (id)	Stores top-level project workspace metadata and creation timestamps
users	id TEXT, display_name TEXT, color TEXT	PRIMARY KEY (id)	Retains collaborator identity and visual presence badge colors
messages	id INTEGER, project_id TEXT, user_id TEXT, role TEXT, content TEXT, timestamp TIMESTAMP	PRIMARY KEY AUTOINCREMENT (id), FOREIGN KEY (project_id)	Append-only sequential conversational history (\mathcal{M}_{\text{conv}}) across human users and the agent
project_memories	id INTEGER, project_id TEXT, category TEXT, key TEXT, value TEXT, updated_at TIMESTAMP	PRIMARY KEY AUTOINCREMENT (id), UNIQUE(project_id, key)	Deduplicated long-term institutional memory (\mathcal{M}_{\text{proj}}) updated via atomic SQL upserts
file_versions	id INTEGER, project_id TEXT, path TEXT, version	PRIMARY KEY AUTOINCREMENT (id), Index on	Tracks monotonic integer versions (V \in \mathbb{N}^+)

Table Name	Column Schema & Data Types	Integrity & Indexing Constraints	Architectural Purpose
	INTEGER, content TEXT, updated_at TIMESTAMP	(project_id, path)	and content snapshots for OCC verification
git_checkpoints	id INTEGER, project_id TEXT, commit_hash TEXT, message TEXT, created_at TIMESTAMP	PRIMARY KEY AUTOINCREMENT (id), FOREIGN KEY (project_id)	Records Git SHA-1 commit checkpoints generated by the agent or developers

Table 3.4: Relational database schema definitions in summit.db

CHAPTER 4: SYSTEM IMPLEMENTATION AND PROJECT DESCRIPTION

4.1 General

Phase I implementation was executed through an iterative, contract-first engineering workflow with strict separation between client-side state management, server-side orchestration, and agent tool execution. First, the relational schema (summit.db) and workspace isolation boundaries were established to guarantee data integrity. Second, the 15 Pydantic WebSocket event schemas and REST endpoints were defined to lock in the communication contract between the backend and frontend. Third, the asynchronous ReAct agent loop (SummitAgent), deterministic context retriever (ContextBuilder), and LiteLLM dispatcher were integrated. Fourth, the React 19 browser workspace combining Monaco Editor, Xterm.js, live previewing, and the OCC diff modal was constructed and verified against a comprehensive 17-test automated test suite.

4.2 Requirement Analysis

System requirements were derived directly from the operational needs of distributed software engineering teams collaborating simultaneously on dynamic codebases with an autonomous coding agent. To eliminate context amnesia and single-user isolation, the system must synchronize multi-user room state in real time, execute autonomous file and shell operations safely inside a sandboxed directory, persist structured architectural knowledge across restarts, and prevent concurrent write collisions. These obligations are formalized into Functional Requirements (Table 4.1) and Non-Functional Requirements (Table 4.2).

4.3 Functional Requirements

Table 4.1 specifies the thirteen core functional requirements (FR1 through FR13) implemented and verified in Phase I.

ID	Functional Requirement Specification
FR1	The system shall support concurrent browser sessions connected to a shared project room via full-duplex WebSockets (/api/projects/{id}/agent/stream).
FR2	The system shall track and broadcast real-time participant presence (USER_JOINED, USER_LEFT) with display names and avatar colors.
FR3	The system shall provide an in-browser code editor (Monaco Editor) supporting syntax highlighting, multi-file navigation, and versioned file saves.
FR4	The system shall execute an autonomous multi-turn ReAct agent loop capable of invoking read_file, write_file, run_terminal, save_memory, and git_checkpoint.
FR5	The system shall broadcast live agent deliberation traces (AGENT_THOUGHT) and tool execution lifecycles (TOOL_START, TOOL_END) to all room peers simultaneously.
FR6	The system shall provide interactive lifecycle controls allowing any connected developer to start, PAUSE, RESUME, or STOP an active agent task.
FR7	The system shall persist append-only multi-user conversational history in the

ID	Functional Requirement Specification
	SQLite messages table across browser reloads and server restarts.
FR8	The system shall persist categorized architectural knowledge (architecture, convention, decision, task) in project_memories using SQL upsert deduplication.

Table 4.1: Functional requirements specification

4.4 Non-Functional Requirements

Table 4.2 defines the non-functional quality attributes governing system latency, transactional reliability, security sandboxing, and extensibility.

Requirement Category	Specification & Operational Target
Performance & Latency	Real-time WebSocket telemetry broadcast latency maintained under 50ms across local network connections; non-blocking ASGI subprocess streaming.
Reliability & ACID Compliance	All conversational logs, memory upserts, and monotonic file version increments executed with transactional ACID integrity in SQLite3.
Security & Workspace Isolation	Strict canonical path validation confining all file reads, file writes, and terminal subprocesses to the designated project_dir, rejecting directory traversal attacks (e.g., ../../etc/passwd) with ValueError.
Modularity & Portability	Decoupled React 19 SPA frontend and

Requirement Category	Specification & Operational Target
	FastAPI backend communicating strictly via typed REST and WebSocket contracts; zero local IDE installation required.
Model Interoperability	Provider-agnostic LLM orchestration via LiteLLM supporting instant switching across OpenAI, Anthropic, Google Gemini, DeepSeek, and local Ollama models.

Table 4.2: Non-functional requirements specification

4.5 Technology Selection

Every technology in the Summit stack was selected to fulfill a specific architectural obligation defined in Tables 4.1 and 4.2 without introducing unnecessary operational complexity. Table 4.3 summarizes the full-stack technology components, and Table 4.4 details the engineering rationale behind each selection.

Subsystem Layer	**Selected Technology**	**Functional Role in Summit**
Frontend Framework	React 19 + TypeScript + Vite	High-performance component rendering, strict static type safety for WebSocket events, and fast HMR bundling.
Code Editor Core	Monaco Editor (@monaco-editor/react)	Industry-standard VS Code editing engine providing syntax highlighting, buffer management, and side-by-side diffing.
Terminal Emulator	Xterm.js (@xterm/xterm)	Browser-based ANSI terminal emulator rendering

Subsystem Layer	Selected Technology	Functional Role in Summit
		interactive subprocess streams on an HTML5 canvas.
Backend API & WebSocket Hub	FastAPI (Python 3.11+ / ASGI)	Asynchronous REST gateway and full-duplex WebSocket coordination engine with Pydantic v2 validation.
LLM Orchestration	LiteLLM	Unified SDK routing multi-turn ReAct prompts and tool calls across cloud APIs and local Ollama instances.
Relational Persistence	SQLite3 (SQLAlchemy / aiosqlite)	Serverless, zero-configuration ACID relational database storing memories, messages, file versions, and checkpoints.
Version Control Subsystem	Git CLI Integration (GitService)	Local repository initialization, working-tree diff generation, and automated commit checkpointing.

Table 4.3: Full-stack technology stack

Technology	Engineering Selection Rationale
React 19 + TypeScript	Enforces compile-time contract matching

Technology	Engineering Selection Rationale
	across all 15 WebSocket event schemas and prevents state race conditions during rapid token streaming.
Monaco Editor	Provides native out-of-the-box support for multi-model diff rendering (DiffEditor), which is essential for interactive HTTP 409 OCC conflict resolution.
Xterm.js	Delivers full pseudo-terminal rendering performance inside the browser without requiring external SSH clients or local terminal windows.
FastAPI (asyncio)	Native Python asyncio support allows concurrent handling of WebSocket broadcasts, file I/O, and asynchronous shell subprocesses within a single lightweight process.
LiteLLM	Eliminates brittle provider-specific tool parsing code by normalizing function-calling payloads across Claude, GPT-4o, Gemini, DeepSeek, and Ollama.
SQLite3	Provides native ON CONFLICT DO UPDATE upsert semantics and transactional file-version locking without the memory overhead of external database servers.

Table 4.4: Technology selection rationale

4.6 System Modules

The Summit platform is partitioned into six cohesive modules, each encapsulating a

well-defined domain responsibility. Table 4.5 summarizes these six modules, followed by detailed architectural descriptions of their internal mechanics.

Module Name	Primary Architectural Responsibility
1. Collaborative Web IDE & Editor Module	Renders the React 19 workspace, Monaco Editor buffers, presence avatars, agent stream feed, and visual OCC diff resolver.
2. WebSocket Session & Room Synchronization Module (SessionManager)	Manages project room rosters, connection lifecycles, room execution locks, and broadcasts 15 typed telemetry events.
3. Autonomous ReAct Agent & Context Retrieval Module (SummitAgent & ContextBuilder)	Assembles path-scored prompts, routes inference via LiteLLM, and executes the multi-turn ReAct tool loop.
4. Dual-Tier Memory & Optimistic Concurrency Module (MemoryExtractor & WorkspaceManager)	Enforces SQLite memory upserts, sandboxed path sanitation, monotonic file versioning, and HTTP 409 conflict checks.
5. Terminal Subprocess & Sandboxed Preview Module (TerminalService)	Runs non-blocking shell commands streaming output to Xterm.js and serves live web application previews inside a sandboxed iframe.
6. Git Version Control & Checkpointing Module (GitService)	Initializes workspace Git repositories, generates unified diffs, and records semantic commit checkpoints.

Table 4.5: System module descriptions and architectural responsibilities

4.6.1 Collaborative Web IDE and Editor Module

Implemented in React 19 and TypeScript, this module organizes the browser viewport into a multi-pane engineering workspace. The left sidebar presents the workspace file tree and persistent project memory inspector; the center pane hosts the tabbed Monaco Editor and toggleable side-by-side DiffEditor; the bottom pane houses the interactive Xterm.js

terminal; and the right pane displays the multiplayer chat feed, real-time agent deliberation stream, and sandboxed live web preview.

4.6.2 WebSocket Session and Room Synchronization Module

SessionManager orchestrates all real-time interactions inside a project room. It maps each project_id to its active WebSocket connections and enforces single-flight agent execution using an asyncio.Lock. All outgoing telemetry passes through an event normalizer that validates payloads against 15 standardized schema types before broadcasting JSON frames to connected peers.

4.6.3 Autonomous ReAct Agent and Context Retrieval Module

When triggered by a user prompt, ContextBuilder queries project_memories, messages, and the project filesystem, scoring candidate files by path and keyword overlap. SummitAgent then initiates the ReAct loop through LiteLLM. At each turn, the module checks for user-issued lifecycle interrupts (PAUSE, RESUME, STOP), dispatches requested tool calls, and streams intermediate thoughts and tool results back to the room.

4.6.4 Dual-Tier Memory and Optimistic Concurrency Module

This module bridges the orchestration tier and summit.db. MemoryExtractor handles structured knowledge persistence across the four memory categories (architecture, convention, decision, task), executing SQLite upserts on (project_id, key) to prevent duplication. Concurrently, WorkspaceManager validates every file path against directory traversal and checks V_{\text{expected}} == V_{\text{current}} on every file write, either incrementing the integer version in file_versions or raising an HTTP 409 Conflict exception.

4.6.5 Terminal Subprocess and Sandboxed Preview Module

TerminalService spawns asynchronous OS subprocesses rooted strictly inside the project's workspace directory. Standard output and error streams are read line-by-line using non-blocking asynchronous readers and emitted as TERMINAL_OUTPUT events.

In parallel, the /preview router serves static and dynamic web entrypoints from the workspace into a sandboxed browser <iframe>.

4.6.6 Git Version Control and Checkpointing Module

GitService wraps the Git CLI to provide programmatic version control inside each isolated workspace. Whenever the agent invokes git_checkpoint or a developer requests a manual snapshot, GitService commits the working tree and persists the commit hash and message in the git_checkpoints table.

4.7 Event Telemetry and Tool Action Mapping

To guarantee deterministic synchronization between the Python backend and all connected React clients, every agent tool invocation and room state transition maps to a specific WebSocket event envelope and workspace action. Table 4.6 documents this mapping across the agent tool suite, lifecycle controls, and telemetry schemas.

Subsystem Category	**Trigger / Tool / Event Identifier**	**System Action & Broadcast Behavior**
Agent Tool	read_file(path)	Validates path inside project_dir, reads UTF-8 file and version V_{\text{current}}, emits TOOL_START and TOOL_END.
Agent Tool	write_file(path, content)	Verifies path safety, writes content, increments version to V_{\text{current}} + 1 in file_versions, emits FILE_CHANGED.
Agent Tool	run_terminal(command)	Spawns async subprocess in workspace root, streams

Subsystem Category	Trigger / Tool / Event Identifier	System Action & Broadcast Behavior
		stdout/stderr via TERMINAL_OUTPUT, returns exit code.
Agent Tool	save_memory(category, key, value)	Upserts record into project_memories (ON CONFLICT DO UPDATE), emits MEMORY_UPDATED to refresh client UI.
Agent Tool	git_checkpoint(message)	Stages workspace changes, creates Git commit, logs hash in git_checkpoints, broadcasts checkpoint status.
Room Presence	USER_JOINED / USER_LEFT	Updates room participant roster in SessionManager and refreshes avatar presence bar across all clients.
Agent Telemetry	AGENT_STATUS / AGENT_THOUGHT	Transitions room state (IDLE, THINKING, EXECUTING, PAUSED, ERROR) and streams live deliberation tokens.
Lifecycle Control	PAUSE / RESUME / STOP	Suspends, resumes, or aborts the active SummitAgent ReAct loop and updates room lock state.
Concurrency Gate	HTTP PUT /files	Rejects stale write with

Subsystem Category	Trigger / Tool / Event Identifier	System Action & Broadcast Behavior
	(V_{\text{expected}} \neq V_{\text{current}})	HTTP 409 Conflict payload and opens the client side-by-side Monaco Diff Modal.

Table 4.6: Mapping of WebSocket events and autonomous agent tools to system actions

4.8 Module Integration

Integrating the six modules requires careful coordination of shared asynchronous resources. When a REST request or WebSocket frame arrives at the FastAPI gateway, SessionManager coordinates with WorkspaceManager, ContextBuilder, SummitAgent, GitService, and TerminalService over a shared asynchronous SQLAlchemy connection pool connected to summit.db. Because multiple developers and the autonomous agent can attempt to mutate the same workspace simultaneously, two arbitration mechanisms govern module integration.

First, **Agent Execution Serialization** is enforced by SessionManager, which maintains one active ReAct loop per project room via an asyncio.Lock, ensuring that concurrent prompts from different teammates do not spawn racing agent loops within the same workspace directory. Second, **File Mutation Arbitration** is enforced across both the REST /files router (used by human editors) and the write_file agent tool by delegating all file persistence to WorkspaceManager. Whenever the agent modifies a file, FILE_CHANGED is broadcast immediately; if a human developer has unsaved local edits on that same file and attempts to save with the prior version integer, the OCC gate catches the version mismatch and launches the visual diff resolver rather than losing either party's work.

4.9 System Configuration

Development, integration, and functional verification of Summit were carried out on standard development hardware. Tables 4.7 and 4.8 specify the hardware and software configurations utilized during Phase I.

Hardware Component	Specification
Processor	Intel Core i5 / i7 or AMD Ryzen 5 / 7 multi-core processor (x86_64 / ARM64)
Memory (RAM)	8 GB RAM minimum (16 GB recommended when running local Ollama models)
Storage	256 GB SSD (for project workspaces, Git objects, and summit.db WAL storage)
Network Interface	Local loopback (127.0.0.1) or LAN / Wi-Fi interface supporting RFC 6455 WebSockets
Client Display & Input	Standard desktop or laptop display capable of running a modern Chromium, Firefox, or WebKit browser

Table 4.7: Hardware configuration for development and verification

Software Component	Specification & Configuration
Operating System	Windows 10 / 11, Linux (Ubuntu 22.04+), or macOS
Backend Runtime	Python 3.11+ with FastAPI, Uvicorn ASGI server, SQLAlchemy, aiosqlite, and LiteLLM
Frontend Build & Runtime	Node.js 20+, React 19, TypeScript 5, Vite dev proxy targeting http://127.0.0.1:8000
Database Engine	SQLite 3 (summit.db) configured via DATABASE_URL in .env
Environment Configuration	.env declaring WORKSPACE_DIR,

Software Component	Specification & Configuration
	DATABASE_URL, model designations, and optional API keys (OPENAI_API_KEY, ANTHROPIC_API_KEY, GEMINI_API_KEY)

Table 4.8: Software and runtime environment configuration

4.10 Feasibility Study

4.10.1 Economical Feasibility

Summit is built entirely on free, permissive open-source frameworks: FastAPI, React 19, TypeScript, Vite, Monaco Editor, Xterm.js, LiteLLM, and SQLite3. Because SQLite operates serverless on disk and ContextBuilder uses deterministic path-relevance scoring rather than paid third-party vector databases, infrastructure hosting costs are near zero. Furthermore, LiteLLM allows student and enterprise teams to route requests to highly economical commercial models (such as DeepSeek-V3 or GPT-4o-mini) or zero-marginal-cost local open-weight models via Ollama.

4.10.2 Technical Feasibility

FastAPI’s native asyncio event loop handles concurrent WebSocket connections and non-blocking subprocess streams with minimal CPU and memory overhead. On the client side, Microsoft’s Monaco Editor and Xterm.js are battle-tested engines deployed in commercial cloud IDEs, guaranteeing responsive rendering inside standard web browsers without custom native plugins.

4.10.3 Social Feasibility

In collaborative software engineering and computer science education, opaque AI code generation often erodes code ownership and hinders junior developer learning. By exposing the AI agent's live deliberation traces, terminal commands, and persistent

architectural memories in a shared multiplayer room, Summit transforms AI coding from an isolated black box into a transparent, educational team workflow.

4.11 Verification and Testing Strategy

Verification of the Phase I implementation was conducted systematically across five testing tiers, progressing from isolated unit validators to multi-client system tests.

4.11.1 Unit Testing

Unit tests isolate individual helper functions, Pydantic schema normalizers, and security validators without external network dependencies. Specifically, test_event_normalizer.py verifies deterministic Pydantic serialization and payload formatting across all 15 WebSocket event envelope types, while test_workspace.py exercises canonical path sanitation in WorkspaceManager, asserting that directory traversal strings (such as ../secret.txt or ../../etc/passwd) immediately raise ValueError.

4.11.2 Integration Testing

Integration tests verify data handoffs and transactional consistency across interacting backend modules. The test_api.py suite exercises the FastAPI REST routers across project creation, file read/write lifecycles, OCC version increments, and terminal subprocess execution. Concurrently, test_agent_memory_integration.py confirms that architectural facts saved via the agent's save_memory tool are atomically upserted into project_memories and automatically retrieved by ContextBuilder into subsequent agent prompt contexts.

4.11.3 Functional Testing

Functional tests validate each obligation in Table 4.1 against end-to-end execution workflows. These tests verify that multi-turn conversations across users and the agent persist accurately to the SQLite messages table, and confirm that valid file edits increment monotonic integer versions (V \rightarrow V+1) and emit FILE_CHANGED notifications whereas stale version IDs reliably trigger HTTP 409 Conflict responses.

4.11.4 System Testing

System tests evaluate the assembled full-stack application under concurrent multi-client conditions. Simultaneous WebSocket clients are connected to /api/projects/{id}/agent/stream to verify synchronized broadcast of USER_JOINED, AGENT_STATUS, AGENT_THOUGHT, TOOL_START, TOOL_END, and USER_LEFT events, while end-to-end shell execution and output streaming are verified through /api/projects/{id}/terminal/execute.

4.11.5 White Box and Black Box Testing

White box testing inspected internal control-flow branching inside context_builder.py, workspace.py, and SessionManager lock acquisition routines. Black box testing issued HTTP REST requests and WebSocket frames with valid, malformed, and conflicting version payloads, verifying exact HTTP status codes (200 OK, 400 Bad Request, 404 Not Found, 409 Conflict) and structured JSON error bodies. Table 4.9 documents the complete 17-test automated verification suite executed against the Phase I implementation, where all 17 tests passed with zero failures.

Test ID	**Test Module / Suite**	**Input / Scenario**	**Expected System Output**	**Status**
TC-01	test_event_normalizer.py	Serialize USER_JOINED & USER_LEFT presence payloads	Validated JSON WebSocket envelope with user metadata	**Pass**
TC-02	test_event_normalizer.py	Serialize AGENT_STATUS & AGENT_THOUGHT frames	Validated status transition and thought stream schema	**Pass**

Test ID	Test Module / Suite	Input / Scenario	Expected System Output	Status
TC-03	test_event_normalizer.py	Serialize TOOL_START & TOOL_END execution payloads	Validated tool name, input arguments, and observation output	**Pass**
TC-04	test_event_normalizer.py	Serialize FILE_CHANGED, TERMINAL_OUTPUT, MEMORY_UPDATED	Validated all 15 typed WebSocket event schemas cleanly	**Pass**
TC-05	test_workspace.py	Read and write valid file path inside project_dir	File persisted to disk; version initialized to V=1	**Pass**
TC-06	test_workspace.py	Attempt relative path traversal (../secret.txt)	Blocked immediately; raises ValueError	**Pass**
TC-07	test_workspace.py	Attempt deep system traversal (../../etc/passwd)	Blocked immediately; workspace isolation preserved	**Pass**
TC-08	test_workspace.py	Sequential file update with matching	Write succeeds; monotonic version	**Pass**

Test ID	Test Module / Suite	Input / Scenario	Expected System Output	Status
		V_{\text{expected}} == V_{\text{current}}	increments to V=2	
TC-09	test_workspace.py	Concurrent file write with stale V_{\text{expected}} < V_{\text{current}}	Write rejected; raises conflict returning server content	**Pass**
TC-10	test_api.py	POST /api/projects project creation & initialization	Returns HTTP 200 with isolated workspace and DB row	**Pass**

Table 4.9: Functional verification test suite (17 automated test cases)

4.12 Initial Results

The Phase I implementation of Summit is complete and functionally verified across all architectural tiers. The React 19 browser workspace renders the Monaco Editor, interactive Xterm.js terminal, sandboxed live web preview, participant presence bar, and side-by-side OCC conflict resolution modal. On the backend, the FastAPI orchestration gateway manages multi-user WebSocket rooms, enforces single-flight ReAct agent locks, executes all five autonomous agent tools (read_file, write_file, run_terminal, save_memory, git_checkpoint), and persists both sequential messages and deduplicated project memories in summit.db.

Execution of the automated pytest verification suite confirmed **17 passed tests and 0 failed tests**. Stale-version file writes reliably return HTTP 409 Conflict responses,

repeated memory writes update existing keys via SQLite upserts without row duplication, and all directory traversal exploits outside the project root are deterministically blocked.

4.13 Challenges

Three primary engineering challenges were encountered and resolved during Phase I development. The first challenge involved **asynchronous WebSocket concurrency**, as managing connection states when developers refresh their browsers or close tabs abruptly required wrapping socket receive loops in structured cleanup blocks that deterministically release room locks and broadcast USER_LEFT presence notifications. The second challenge involved **LLM tool call parsing**, where inconsistent function-calling payload formats across diverse model providers were resolved by standardizing all inference routing through LiteLLM's unified schema combined with Pydantic validation. The third challenge involved **subprocess streaming across operating systems**, where non-blocking standard output reading on Windows and POSIX environments required specialized asynchronous stream reader coroutines to prevent blocking the FastAPI event loop.

CHAPTER 5: CONCLUSIONS AND FUTURE ENHANCEMENTS

5.1 Conclusion

Summit demonstrates a cohesive, full-stack architecture that transforms AI-assisted software development from an isolated, stateless single-user utility into a collaborative, memory-persistent team discipline. By uniting a browser-native Web IDE (Monaco Editor, Xterm.js terminal, and sandboxed live preview) with a FastAPI multiplayer WebSocket synchronization hub, a LiteLLM-routed ReAct agent loop, a dual-tier SQLite memory store, and Optimistic Concurrency Control (HTTP 409 with visual diff resolution), Summit directly resolves the episodic-temporal deficit and single-user isolation paradigm that limit contemporary developer tools.

5.2 Current Status

At the conclusion of Phase I, the core system architecture is fully operational across the browser-native React 19 Web IDE, the FastAPI orchestration backend, and the ACID-compliant SQLite relational database (summit.db). The multiplayer WebSocket synchronization engine (SessionManager) has been validated across concurrent client connections, streaming 15 standardized telemetry event types and maintaining live room presence rosters. The multi-turn ReAct autonomous agent (SummitAgent) actively interacts with isolated local project workspaces through five core tools (read_file, write_file, run_terminal, save_memory, git_checkpoint) while ContextBuilder injects persistent memories and path-scored files into every prompt. All 17 automated functional verification tests across unit, integration, and API suites pass cleanly with zero failures.

5.3 Limitations

While Phase I establishes a robust, working collaborative platform, three architectural boundaries remain for Phase II expansion. First, **file-level concurrency** is enforced at the file save boundary via OCC (HTTP 409), meaning that concurrent character-by-character typing by two users on adjacent lines of the same open file buffer requires manual diff

resolution upon save. Second, **context window token limits** are currently managed via deterministic path-relevance scoring in ContextBuilder, which is fast and dependency-free for small-to-medium workspaces but requires semantic vector pruning when scaling to large repositories with hundreds of files. Third, **host subprocess execution** runs shell commands within path-validated local host directories rather than inside kernel-isolated disposable containers.

5.4 Future Objectives

To address the boundaries identified in Section 5.3 and transition Summit toward production-scale deployment, Phase II is structured around five core engineering objectives. The first objective is **keystroke-level real-time co-editing**, integrating Conflict-Free Replicated Data Types (Yjs or Automerge) directly into Monaco Editor buffers over WebSockets to enable concurrent character-by-character pair programming and live remote cursor rendering between humans and the AI agent. The second objective is **semantic vector search (ChromaDB integration)**, augmenting relational project memory with dense vector embeddings to support hybrid semantic codebase retrieval across large repositories. The third objective is a **sandboxed container runtime**, encapsulating terminal command execution within isolated Docker containers or WebAssembly (Wasm) runtimes to guarantee complete host OS security. The fourth objective is **multi-agent team orchestration**, coordinating specialized sub-agents within the same collaborative room. The fifth objective is **comprehensive benchmark evaluation**, measuring end-to-end multi-client latency, CRDT merge overhead, and multi-turn task resolution rates.

5.5 Future Enhancements

Beyond the core objectives, Summit's modular WebSocket and LiteLLM foundation supports three high-value future extensions. First, **voice and multimodal prompting** will allow developers to speak architectural instructions directly into the collaborative room with automatic transcription and intent dispatch alongside uploaded UI wireframes.

Second, **automated pull request synthesis** will enable GitService and SummitAgent to package multi-file features into dedicated Git branches and automatically draft complete GitHub or GitLab pull requests with architectural summary notes. Third, **multi-agent specialization** will allow multiple specialized sub-agents—such as an Architect Agent, a Test Generator Agent, and a Code Reviewer Agent—to collaborate simultaneously inside the shared project room.

CO–SDG RELEVANCE RECORD

The Course Outcomes (COs) of the project **"SUMMIT: A COLLABORATIVE MULTIPLAYER WEB IDE AND AI CODING AGENT WITH PERSISTENT PROJECT MEMORY"**, executed by batch members **MUHILAN S(241701033)** and **DHANESH SK(241701013)** under the supervision of **Dr. P. Revathy, Ph.D.** (Professor, Department of Computer Science and Design, Rajalakshmi Engineering College), address the United Nations Sustainable Development Goals (SDGs) as documented below.

SDG (No. & Theme)	**Addressed COs**	**Topic / Activity Addressing SDG Theme**
SDG 4 – Quality Education	CO1, CO3	Democratizes collaborative programming education by exposing live AI agent deliberation traces, terminal commands, and architectural conventions in a shared browser room requiring zero local setup.
SDG 8 – Decent Work and	CO2, CO5	Enhances software

SDG (No. & Theme)	Addressed COs	Topic / Activity Addressing SDG Theme
Economic Growth		engineering productivity and team alignment while preventing destructive write collisions through Optimistic Concurrency Control and automated context assembly.
SDG 9 – Industry, Innovation and Infrastructure	CO2, CO4	Advances cloud software engineering infrastructure by integrating full-duplex WebSocket rooms, multi-provider LiteLLM routing, and dual-tier ACID relational memory into a unified Web IDE.
SDG 10 – Reduced Inequalities	CO1, CO3	Reduces hardware and economic barriers to advanced AI tooling by running entirely inside a standard web browser and supporting zero-cost local open-weight models via Ollama alongside commercial APIs.

COURSE OUTCOMES

Course Outcome	Statement of Outcome
CO1	Analyse the architectural limitations of stateless, single-user AI coding assistants and formulate the system requirements for a collaborative multiplayer Web IDE with persistent project memory.
CO2	Design and implement a browser-native workspace integrating Microsoft's Monaco Editor, an interactive Xterm.js terminal emulator, sandboxed iframe web previewing, and full-duplex FastAPI WebSocket room synchronization.
CO3	Construct an ACID-compliant dual-tier relational memory architecture in SQLite (messages and project_memories) paired with a deterministic path-relevance ContextBuilder to eliminate cross-session context amnesia.
CO4	Integrate an autonomous multi-turn ReAct coding agent routed via LiteLLM with five sandboxed workspace tools (read_file, write_file, run_terminal, save_memory, git_checkpoint) and real-time lifecycle steering controls.

PROGRAM OUTCOMES

PO	Outcome	Description
PO1	**Engineering Knowledge**	Apply knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
PO2	**Problem Analysis**	Identify, formulate, review research literature and analyse complex engineering problems to reach substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.
PO3	**Design / Development of Solutions**	Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate

PO	Outcome	Description
		consideration of public health and safety, and cultural, societal and environmental considerations.
PO4	**Conduct Investigations of Complex Problems**	Use research-based knowledge and research methods, including design of experiments, analysis and interpretation of data and synthesis of information, to provide valid conclusions.
PO5	**Modern Tool Usage**	Create, select and apply appropriate techniques, resources and modern engineering and IT tools, including prediction and modelling, to complex engineering activities with an understanding of the limitations.
PO6	**The Engineer and Society**	Apply analytical judgment informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional

PO	Outcome	Description
		engineering practice.
PO7	**Environment and Sustainability**	Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of, and need for, sustainable development.
PO8	**Ethics**	Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
PO9	**Individual and Team Work**	Function effectively as an individual, and as a member or leader in diverse teams

PROGRAM SPECIFIC OUTCOMES

PSO	**Area**	**Description**
PSO1	**Computing Foundations**	Strength in programming languages and competence in computing technologies and tools to design and implement efficient software solutions using suitable algorithms, data structures

PSO	Area	Description
		and computing techniques.
PSO2	**HCI and Digital Experience Design**	Independently investigate problems through a human-computer interaction design process and develop end-to-end solutions from user needs to interface design, coding and evaluation.
PSO3	**Research, Innovation and Professional Growth**	Apply knowledge across domains to identify research gaps and develop new ideas, pursue higher studies or innovative career paths, and evolve as an ethically responsible computer science and design professional.
PSO4	**Inclusive and Accessible System Design**	Design and evaluate interactive computing systems that remain usable by people with differing physical abilities and hardware environments, applying inclusive-design principles to the interfaces and applications built.

CO–PO–PSO MAPPING

The correlation matrix below maps the five Course Outcomes (CO1–CO5) of the Summit platform against the twelve Program Outcomes (PO1–PO12) and four Program Specific Outcomes (PSO1–PSO4) for batch members **MUHILAN S(241701033)** and **DHANESH SK (241701013)** under the supervision of **Dr. P. Revathy, Ph.D.**.

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3	PSO4
CO1	3	3	3	3	3	2	2	2	--	2	--	2	3	3	2	2
CO2	3	3	3	2	3	2	2	2	3	2	--	2	3	3	2	2
CO3	3	3	3	3	3	2	2	2	3	--	2	2	3	2	3	--
CO4	3	3	3	3	3	3	2	3	3	3	3	3	3	3	3	2
CO5	2	2	3	3	3	3	2	3	3	3	3	3	3	3	3	2

Correlation Scale: 1 – Slight (Low); 2 – Moderate (Medium); 3 – Substantial (High); "--" – No direct correlation.

Works cited

1. The Impact of AI on Developer Productivity: Evidence from GitHub, https://www.semanticscholar.org/paper/The-Impact-of-AI-on-Developer-Productivity%3A-from-Peng-Kalliamvakou/15abedb29536d50afeeec739a25358255cbda3e8 2. AI Tools Make Programmers More Productive - NN/G, https://www.nngroup.com/articles/ai-programmers-productive/ 3. The Programmer's Assistant: Conversational Interaction with a Large, https://arxiv.org/pdf/2302.07080 4. Repository-Level Prompt Generation for Large Language Models of, https://arxiv.org/pdf/2206.12839 5. RepoCoder: Repository-Level Code Completion Through Iterative, https://www.chapterpal.com/s/b414ub8i/repocoder-repository-level-code-completion-through-iterative-retrieval-and-generation